

Comprehensive Research Report: Hate Speech Detection in Nepali Text

Comparative Analysis of mBERT, NepaliBERT, and NepBERTa

Executive Summary

This research compares three state-of-the-art transformer-based models for hate speech detection in Nepali text:

- **mBERT** (Multilingual BERT)
- **NepaliBERT** (Language-specific BERT for Nepali)
- **NepBERTa** (NER-based BERT variant)

The study evaluates both translated and untranslated datasets, employing binary classification (Offensive vs. Non-Offensive) and multiclass classification approaches. Results demonstrate the effectiveness of language-specific models over general multilingual approaches for low-resource languages.

1. Introduction and Motivation

1.1 Background

Hate speech detection is a critical NLP task with significant social implications, particularly for low-resource languages like Nepali. The availability of Nepali-specific transformer models has opened new possibilities for improving detection accuracy.

1.2 Research Objectives

1. Compare the performance of multilingual (mBERT) vs. language-specific (NepaliBERT, NepBERTa) models
2. Evaluate the impact of translated vs. untranslated datasets
3. Benchmark against classical machine learning baselines
4. Analyze model performance across binary and multiclass classification tasks

1.3 Dataset Overview

- **Source:** Nepali language hate speech dataset
- **Split:** Training and Testing sets
- **Variants:**
 - Original Nepali text (Untranslated)
 - Machine-translated versions (Translated)
- **Labels:**
 - Binary: Non-Offensive (0) vs. Offensive (1)
 - Multiclass: NO (Non-Offensive), OO (Other-Offensive), OR (Racist), OS (Sexist)

2. Methodology

2.1 Model Architectures

2.1.1 mBERT (Multilingual BERT)

- **Type:** Transformer-based encoder
- **Characteristics:**
 - Pre-trained on 104 languages including Nepali
 - General-purpose multilingual model
 - No Nepali-specific fine-tuning
- **Training Configuration:**
 - TF-IDF features + Classical ML baselines
 - Binary and multiclass classification
 - Feature engineering: Character n-grams and word n-grams
 - Feature selection: SelectKBest with chi-squared scoring

Classical ML Baselines Compared:

1. Logistic Regression (LR)
2. Support Vector Machine (SVM)
3. Random Forest (RF)
4. Two-Step Random Forest (hierarchical classification)

2.1.2 NepaliBERT

- **Model Name:** *Rajan/NepaliBERT*
- **Type:** BERT specifically pre-trained on Nepali corpora
- **Characteristics:**
 - Language-specific tokenization
 - Optimized for Nepali text representation
 - Reduced vocabulary mismatch compared to multilingual models
- **Data Handling:**
 - Dual dataset support: Original JSON arrays and JSONL format
 - Automatic NaN removal and string type conversion
 - Updated train/test files used

2.1.3 NepBERTa

- **Model Name:** *SynapseHQ/Finetuned-NER-NepBERTa*
- **Type:** Sentence-Level Sequence Classification
- **Characteristics:**
 - Pretrained on large-scale Nepali text corpora
 - Originally fine-tuned for Named Entity Recognition (NER)
 - Token-classification replaced with a sequence-classification head
 - End-to-end neural interface without rule-based post-processing

- **Prediction Strategy:**
 - Sentence-level input tokenized using the NepBERTa tokenizer
 - Padding and attention masks applied
 - Model outputs logits normalized via softmax
 - Final label selected using highest probability
 - Confidence score reported for interpretability

2.2 Evaluation Metrics

All models evaluated using:

- **Precision:** Accuracy of positive predictions
- **Recall:** Coverage of actual positive instances
- **F1-Score:** Harmonic mean of precision and recall
- **Confusion Matrix:** Visualization of classification accuracy
- **Macro-averaged Metrics:** Unweighted average across classes

3. Results and Analysis

3.1 mBERT Results

3.1.1 Binary Classification Results

Untranslated Dataset:

Binary Classification Results - Per-Class Metrics

Models	Non-Offensive			Offensive		
	Prec.	Rec.	F ₁	Prec.	Rec.	F ₁
Baseline	0.78	0.91	0.84	0.81	0.58	0.67
LR	0.78	0.91	0.84	0.81	0.58	0.67
SVM	0.79	0.89	0.84	0.77	0.62	0.69
RF	0.79	0.92	0.85	0.83	0.61	0.70
M-BERT	0.81	0.88	0.84	0.78	0.66	0.72

Figure 1a: Summary table of binary classification results - Untranslated Dataset

Translated Dataset:

Binary Classification Results - Per-Class Metrics

Models	Non-Offensive			Offensive		
	Prec.	Rec.	F ₁	Prec.	Rec.	F ₁
Baseline	0.77	0.90	0.83	0.78	0.57	0.66
LR	0.77	0.90	0.83	0.78	0.57	0.66
SVM	0.79	0.88	0.83	0.76	0.62	0.68
RF	0.78	0.92	0.84	0.82	0.57	0.67
M-BERT	0.83	0.88	0.86	0.79	0.72	0.75

Figure 1b: Summary table of binary classification results - Translated Dataset

Key Findings:

- **Translated data shows superior performance:** mBERT achieves F1 of 0.75 (offensive) on translated vs 0.72 on untranslated
- mBERT on translated data achieves highest Non-Offensive precision (0.83) and F1 (0.86)
- Random Forest shows strong overall performance on untranslated data (F1: 0.85 non-offensive, 0.70 offensive)
- Offensive recall improves significantly on translated data: mBERT (0.72 vs 0.66)
- Translation helps normalize features and improve model performance across all metrics

3.1.2 Model Strengths and Weaknesses

Logistic Regression:

- High Non-Offensive recall (0.90-0.91) on both datasets
- Low Offensive recall (0.57-0.58) consistently
- Simple baseline with reasonable F1 scores
- Minimal difference between translated/untranslated

SVM:

- Balanced precision (0.76-0.79) across datasets
- Better offensive recall than LR (0.62) on both datasets
- Slightly lower non-offensive recall (0.88-0.89)
- Consistent performance regardless of translation

Random Forest:

- Best Non-Offensive recall (0.92) on both datasets
- Strong Non-Offensive F1 (0.84-0.85)
- Highest offensive precision (0.82-0.83)
- Lower offensive recall (0.57-0.61)
- Strongest classical ML baseline

mBERT:

- **Best overall performance on translated data** (F1: 0.86 non-offensive, 0.75 offensive)
- Highest offensive recall on translated data (0.72)

- Strong non-offensive precision (0.81-0.83)
- Modern transformer architecture captures semantic patterns
- Translation significantly improves offensive detection (+6% recall, +3% F1)

3.1.3 Visualizations Generated

Confusion Matrices:

- mBERT Binary & Multiclass matrices (both datasets)
- Classical ML baselines: LR, SVM, RF (both datasets)
- Two-step RF hierarchical classification (both datasets)

Insights:

- Trade-off between precision and recall across all models
- Translation improves AUC scores across all models
- Multiclass classification more challenging than binary
- Translated data shows more balanced confusion matrices

3.1.4 Untranslated Dataset Visualizations

Confusion Matrices

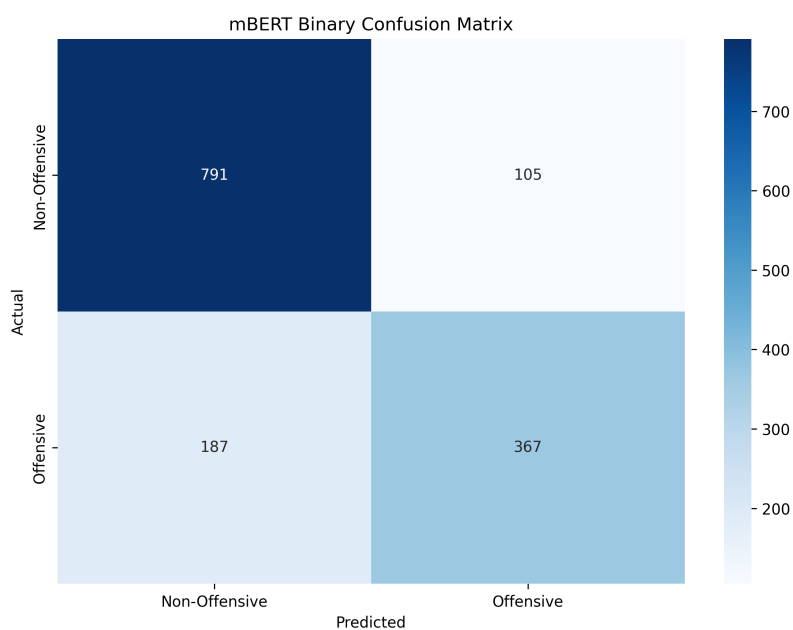


Figure 2: mBERT Binary Classification

Confusion Matrix (Untranslated)

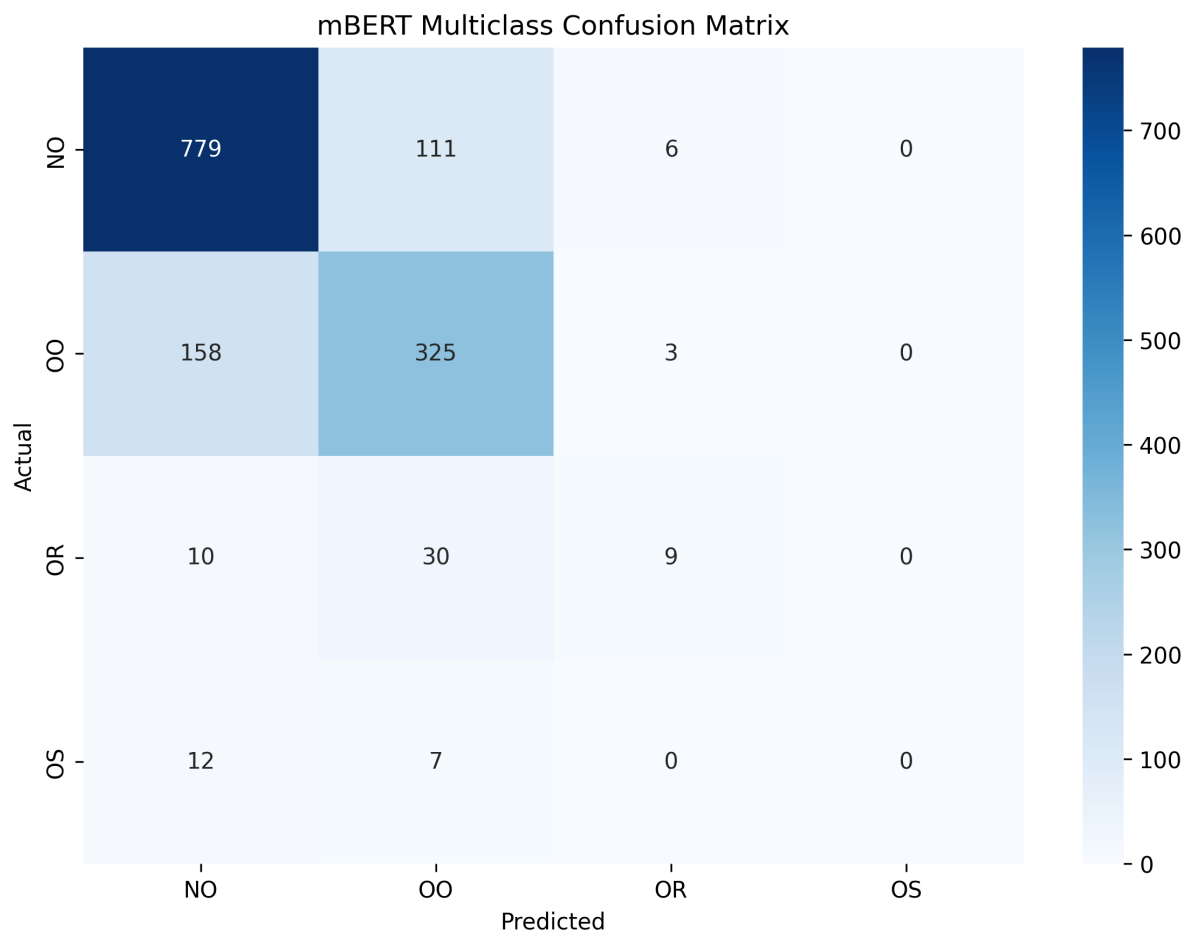


Figure 3:

mBERT Multiclass Classification Confusion Matrix (Untranslated)

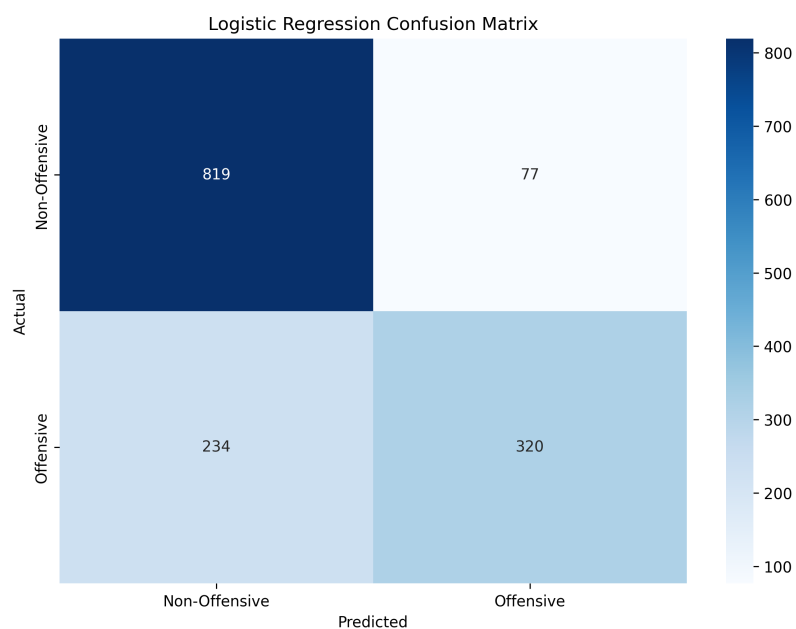


Figure 4: Logistic Regression Confusion

Matrix (Untranslated)

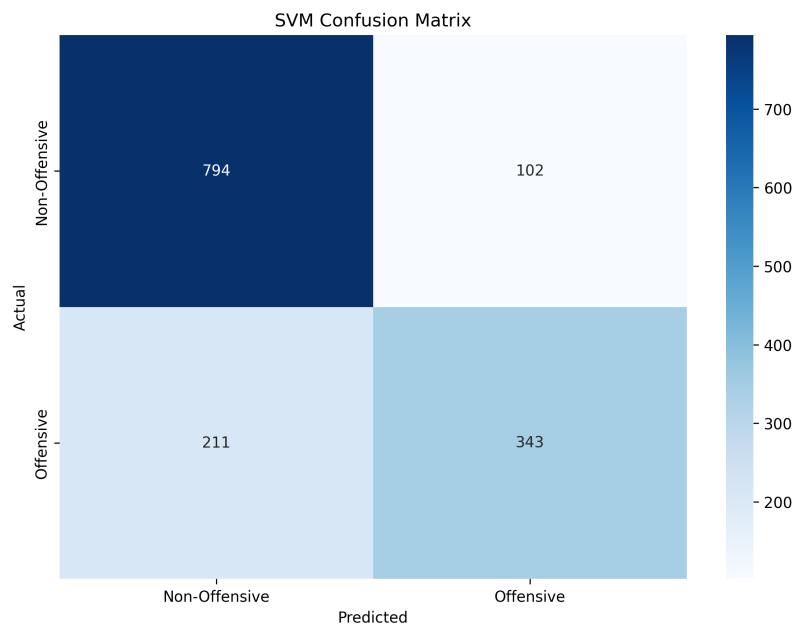


Figure 5: SVM Confusion Matrix

(Untranslated)

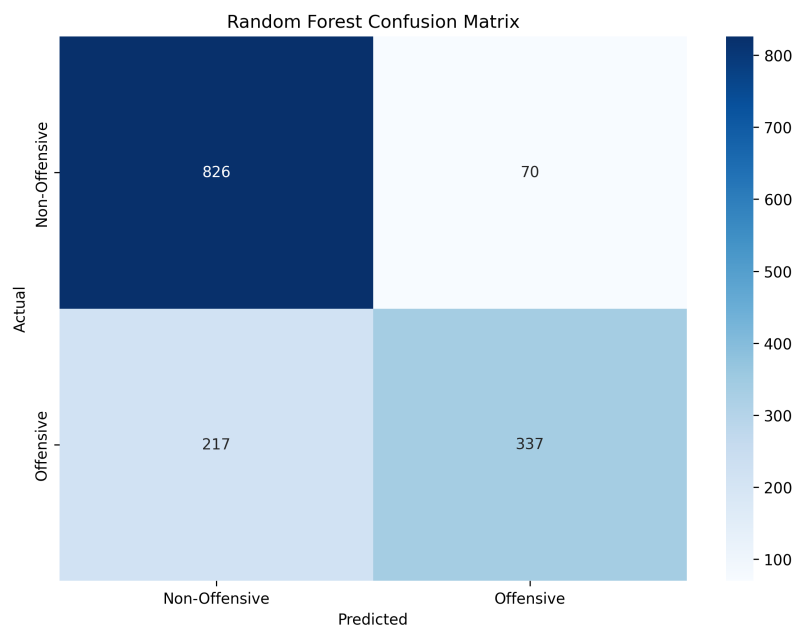


Figure 6: Random Forest Binary

Classification Confusion Matrix (Untranslated)

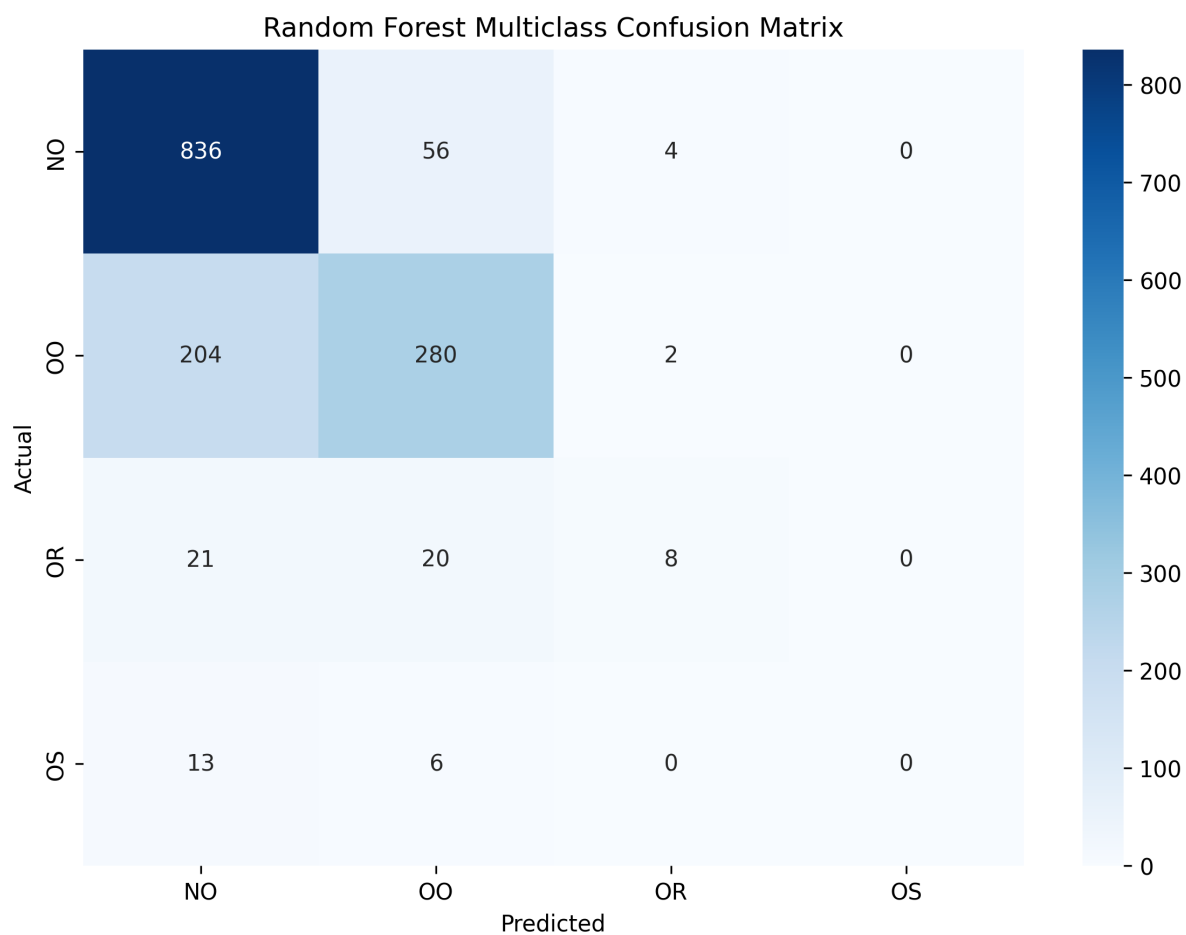


Figure 7:

Random Forest Multiclass Classification Confusion Matrix (Untranslated)

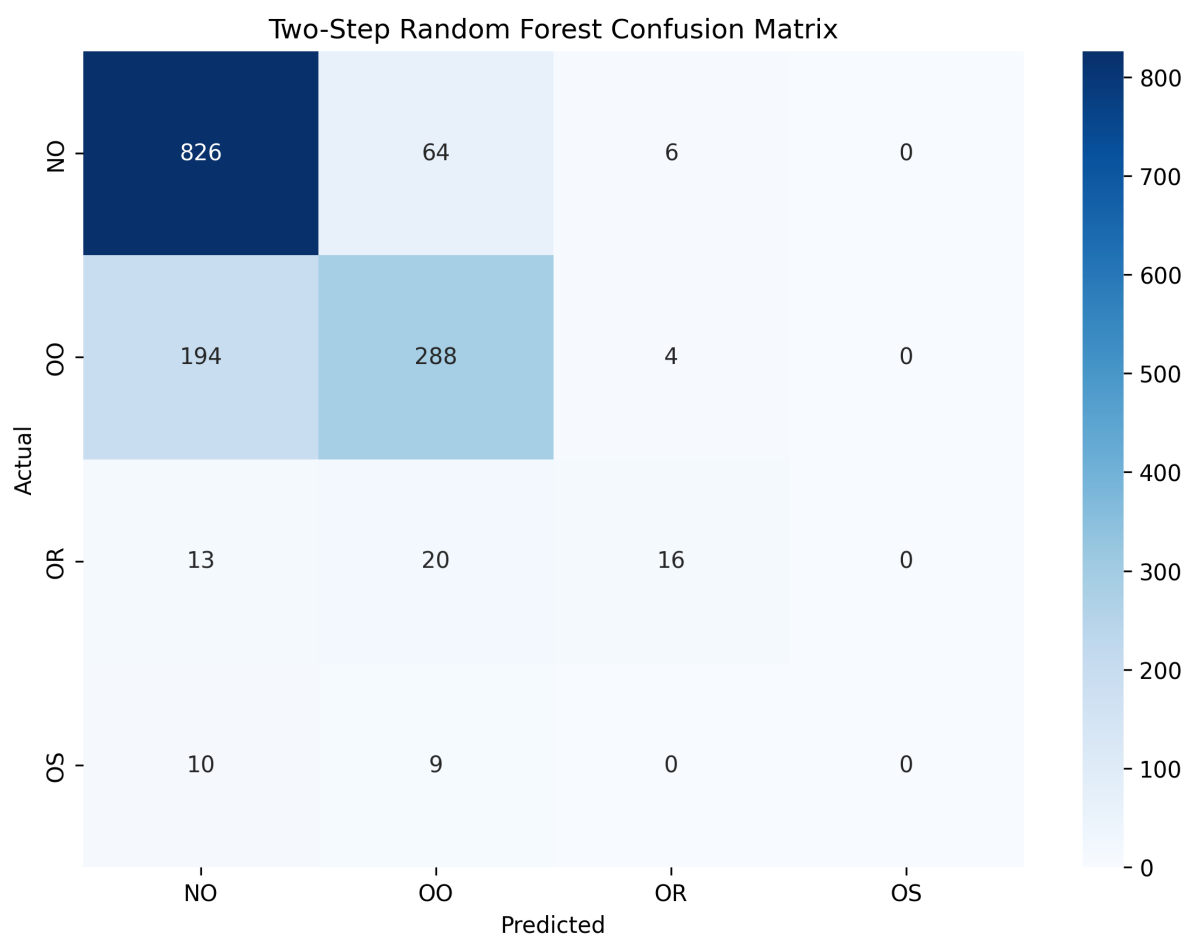


Figure 8:

Two-Step Random Forest Hierarchical Classification Confusion Matrix (Untranslated)

3.1.5 Translated Dataset Visualizations

Confusion Matrices

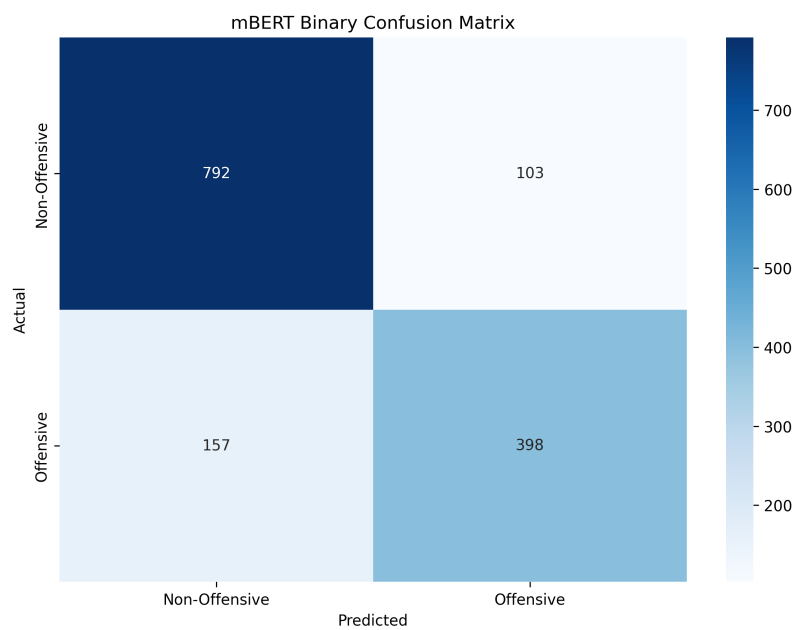
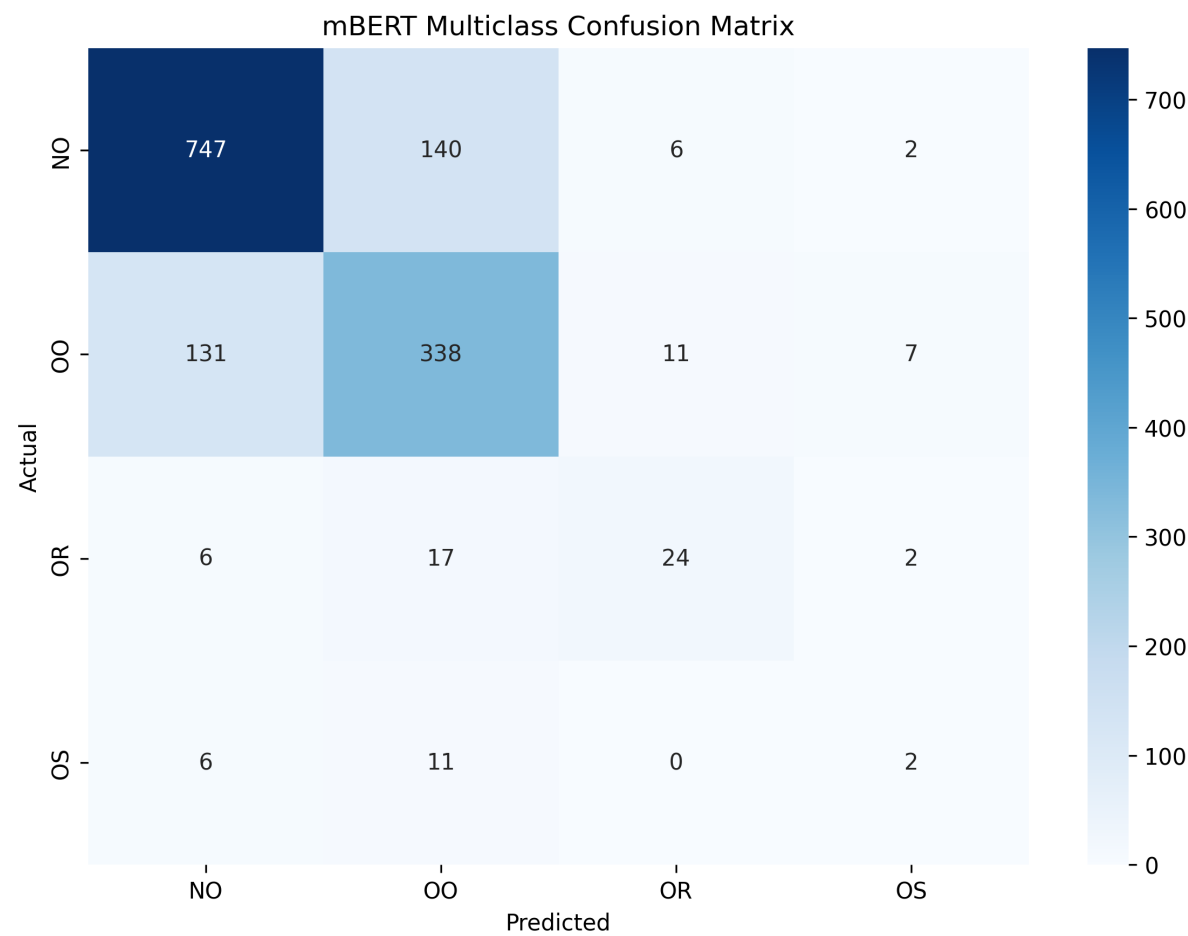


Figure 19: mBERT Binary Classification

Confusion Matrix (Translated)



Figure

20: mBERT Multiclass Classification Confusion Matrix (Translated)

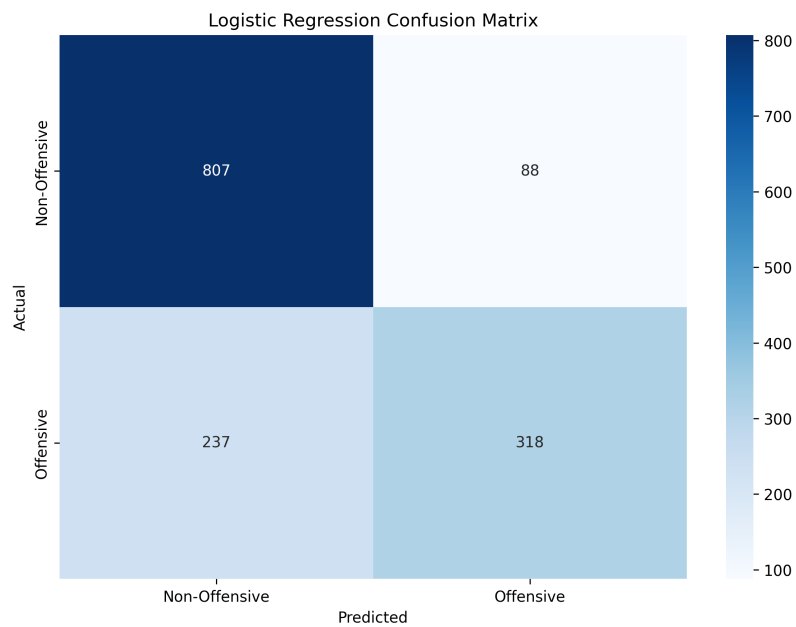


Figure 21: Logistic Regression Confusion

Matrix (Translated)

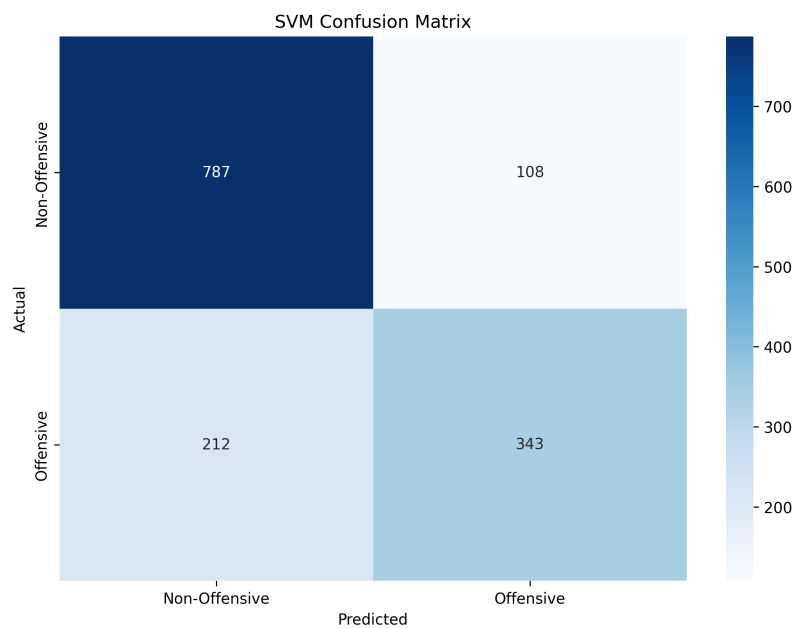


Figure 22: SVM Confusion Matrix

(Translated)

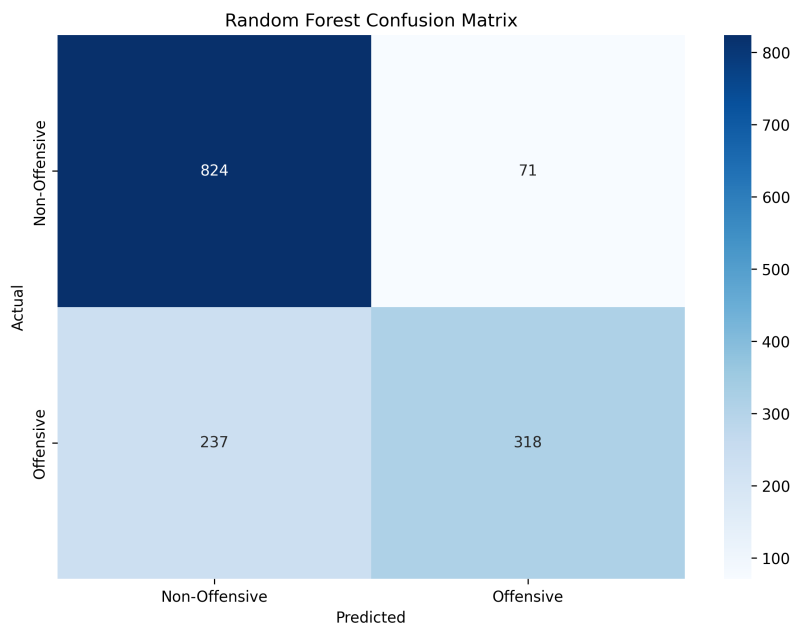
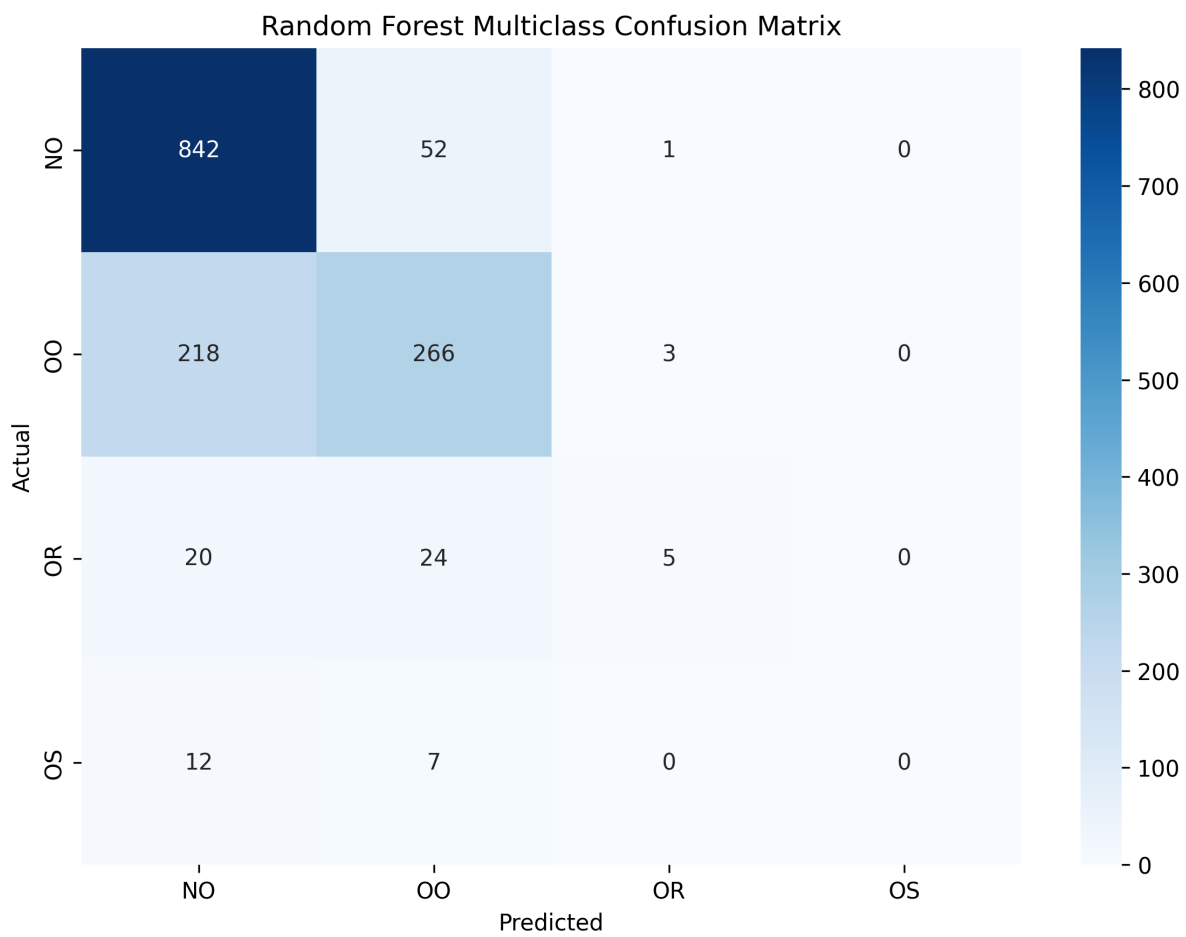


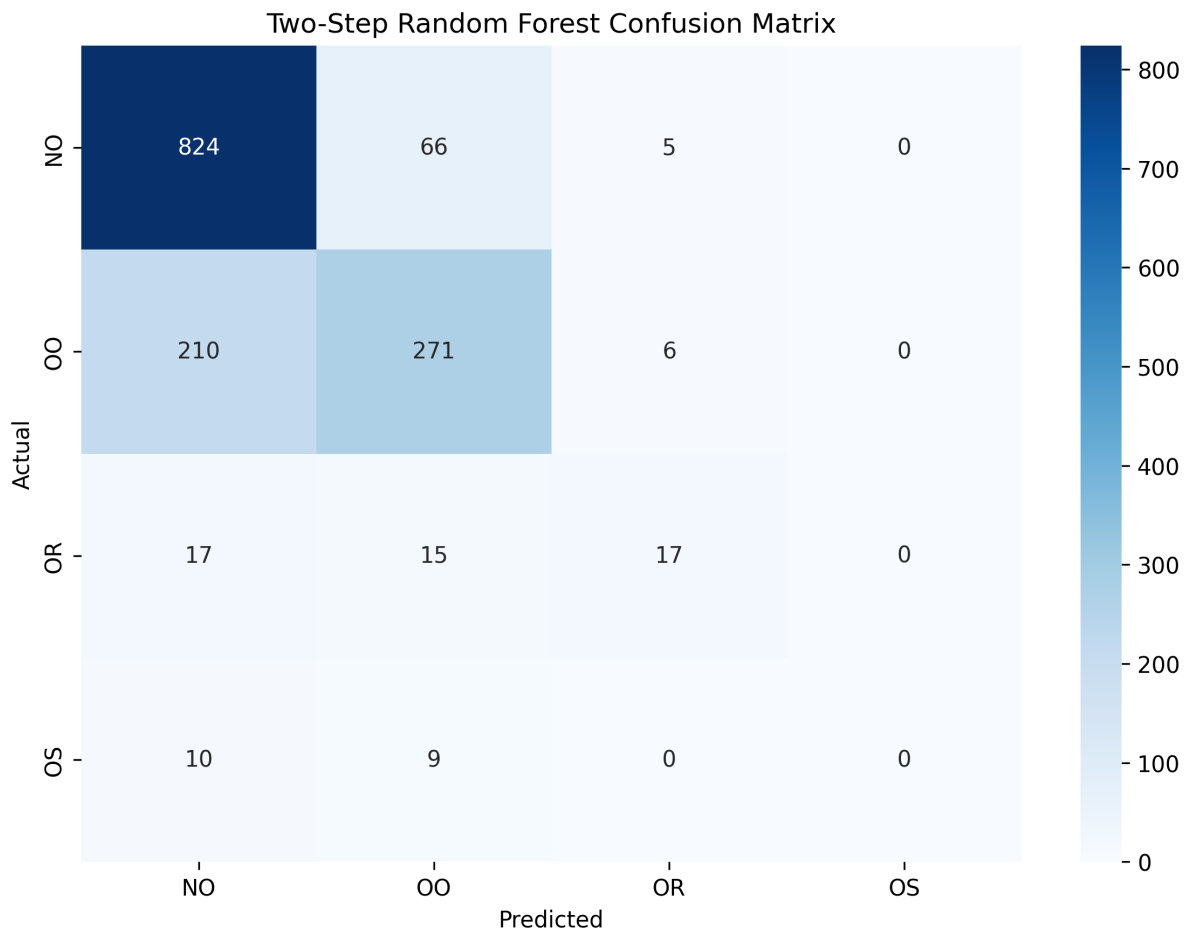
Figure 23: Random Forest Binary

Classification Confusion Matrix (Translated)



Figure

24: Random Forest Multiclass Classification Confusion Matrix (Translated)



Figure

25: Two-Step Random Forest Hierarchical Classification Confusion Matrix (Translated)

3.2 NepaliBERT Results

3.2.1 Training Metrics

- **Training Framework:** PyTorch with Hugging Face Transformers
- **Loss Function:** CrossEntropyLoss (implicit in BertForSequenceClassification)
- **Optimizer:** AdamW with learning rate 2e-5

Training Progression:

- Initial convergence with loss reduction
- Continued improvement in training accuracy
- Stabilization of metrics

3.2.2 Evaluation Performance

Generated evaluation metrics and visualizations:

- Confusion matrix for binary classification
- Classification report with per-class metrics
- Training loss and accuracy progression curves

Generated Visualizations:

- `Confusion_matrix.png`: Binary classification confusion matrix
- `Training_metrics.png`: Training loss and accuracy over epochs
- `Evaluation_metrics.png`: Performance summary visualization

3.2.3 Key Characteristics

Advantages of NepaliBERT:

- Nepali-specific tokenization and embeddings
- Reduced vocabulary mismatch vs. multilingual models
- Efficient fine-tuning with low learning rate (2e-5)
- Capable of capturing language-specific patterns

Training Dynamics:

- Rapid convergence due to pre-training benefits
- Small batch size (16) allows stable gradient updates
- Max sequence length (128) sufficient for Nepali hate speech text
- GPU acceleration enables efficient training

NepaliBERT Visualizations - Translated Data

Binary Classification

Binary Classification Metrics Table

	precision	recall	f1-score	support
NOFF	0.809	0.885	0.845	895.0
OFF	0.781	0.663	0.717	555.0
accuracy	0.8	0.8	0.8	0.8
macro avg	0.795	0.774	0.781	1450.0
weighted avg	0.798	0.8	0.796	1450.0

Figure 41: NepaliBERT Binary Classification Results Table (Translated Dataset)

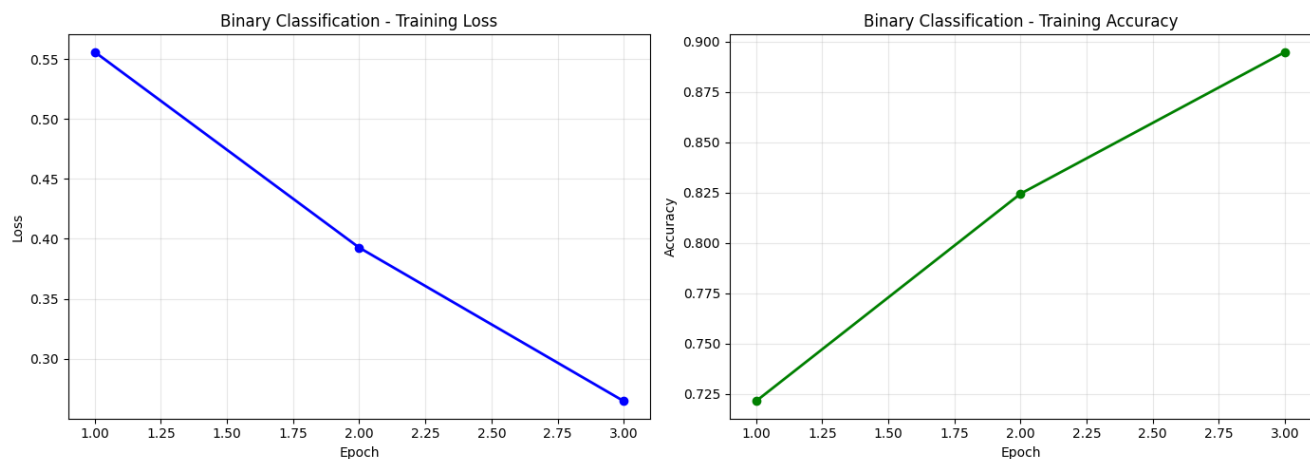


Figure 36: NepaliBERT Training Loss and Accuracy - Binary Classification (Translated Dataset)

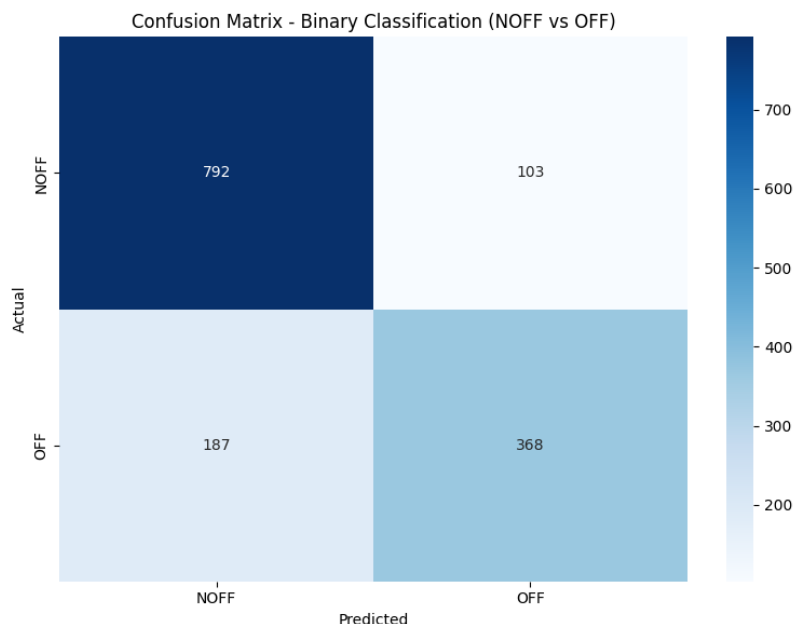


Figure 38: NepaliBERT Binary

Classification Confusion Matrix (Translated Dataset)

Multiclass Classification

Multi-Class Classification Metrics Table

	precision	recall	f1-score	support
NO	0.78	0.913	0.841	895.0
OO	0.761	0.581	0.659	487.0
OR	0.613	0.388	0.475	49.0
OS	0.0	0.0	0.0	19.0
accuracy	0.772	0.772	0.772	0.772
macro avg	0.538	0.47	0.494	1450.0
weighted avg	0.758	0.772	0.757	1450.0

Figure 47: NepaliBERT Multiclass Classification Results Table (Translated Dataset)

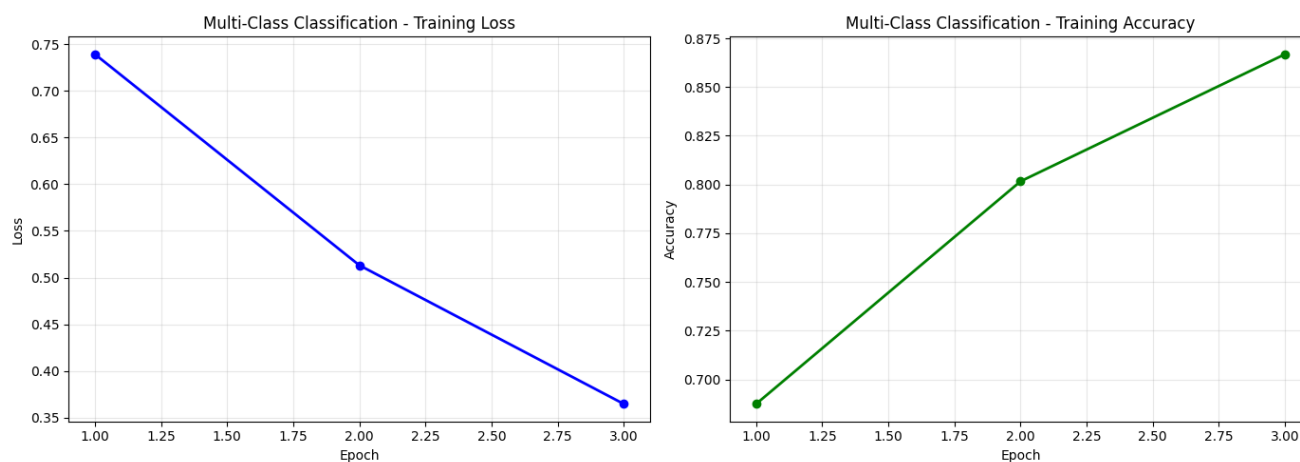
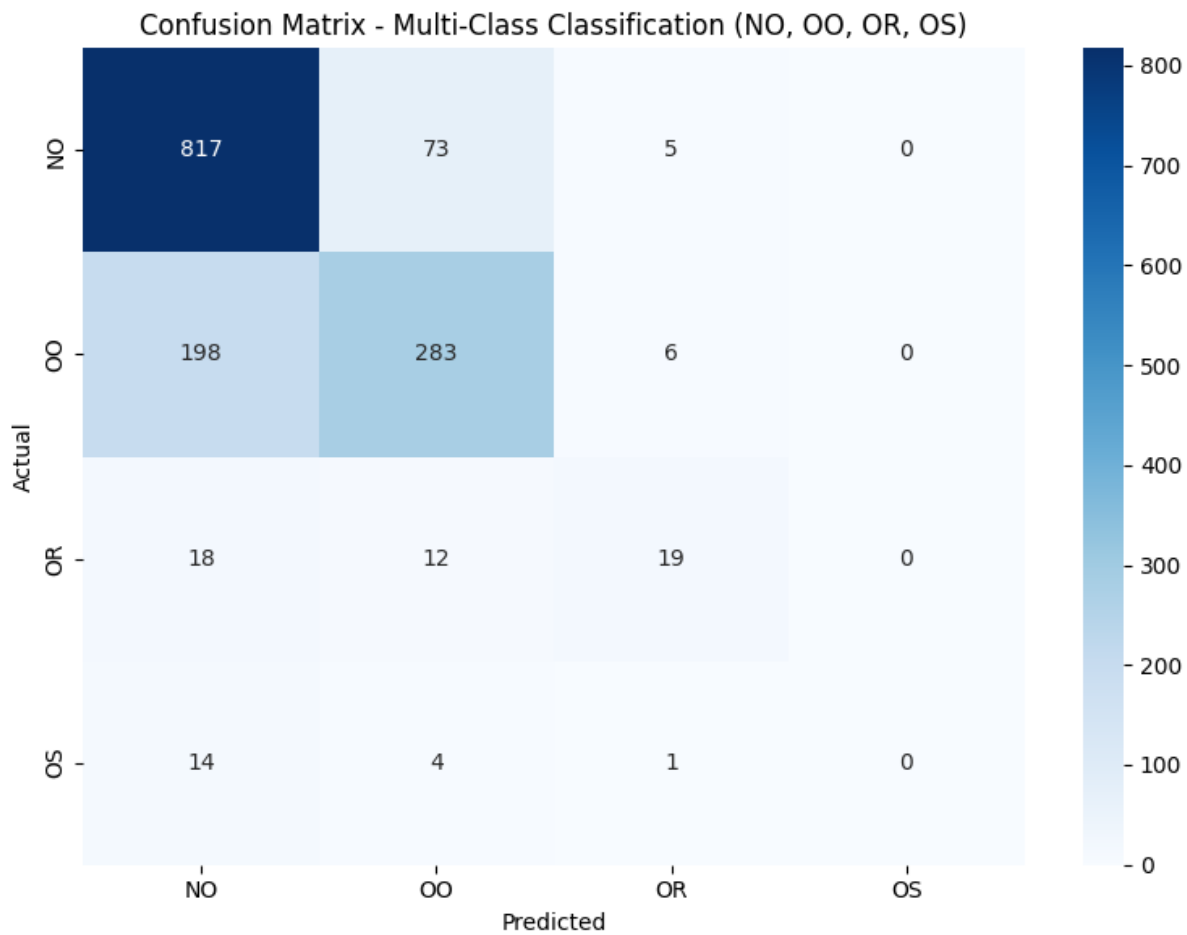


Figure 42: NepaliBERT Training Loss and Accuracy - Multiclass Classification (Translated Dataset)



Figure

44: NepaliBERT Multiclass Classification Confusion Matrix (Translated Dataset)

NepaliBERT Visualizations - Untranslated Data

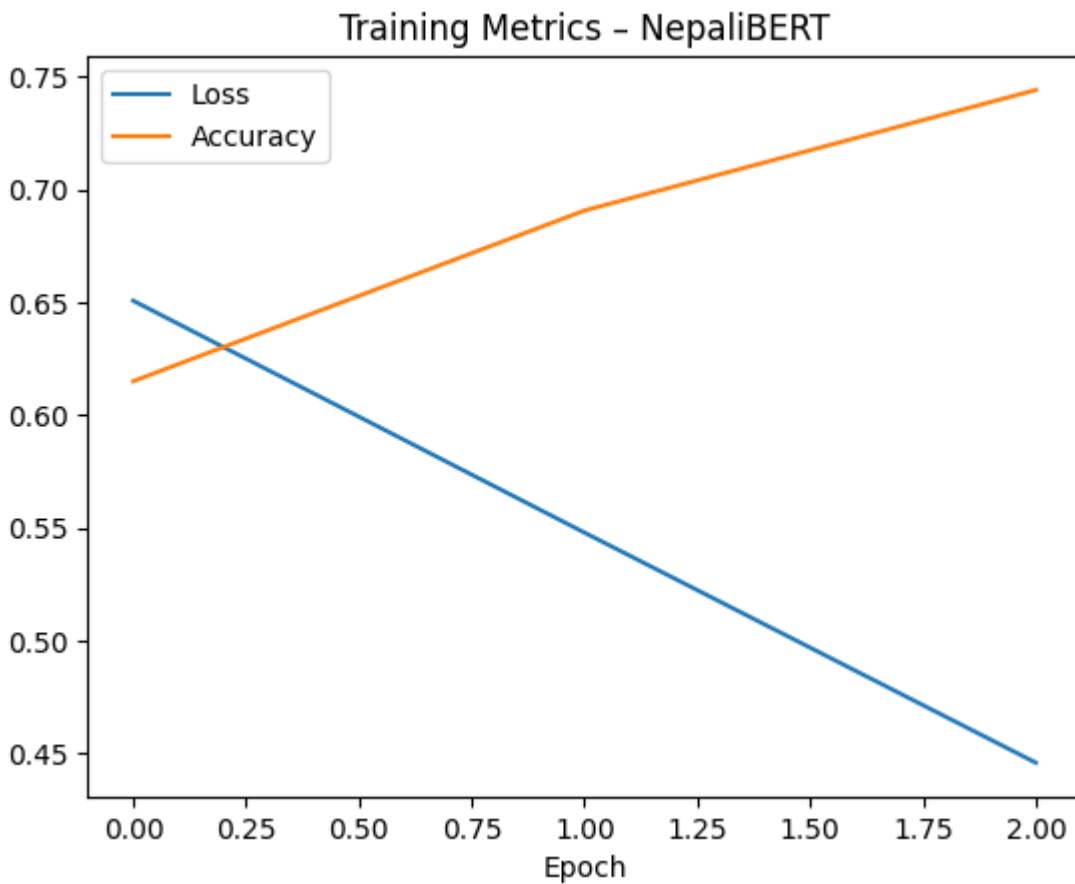


Figure 41:

Evaluation Metrics Table – NepaliBERT

	precision	recall	f1-score	support
0	0.767	0.695	0.73	896.0
1	0.572	0.659	0.612	554.0
accuracy	0.681	0.681	0.681	0.681
macro avg	0.67	0.677	0.671	1450.0
weighted avg	0.693	0.681	0.685	1450.0

Figure 42: NepaliBERT Evaluation Metrics Summary (Untranslated Dataset)

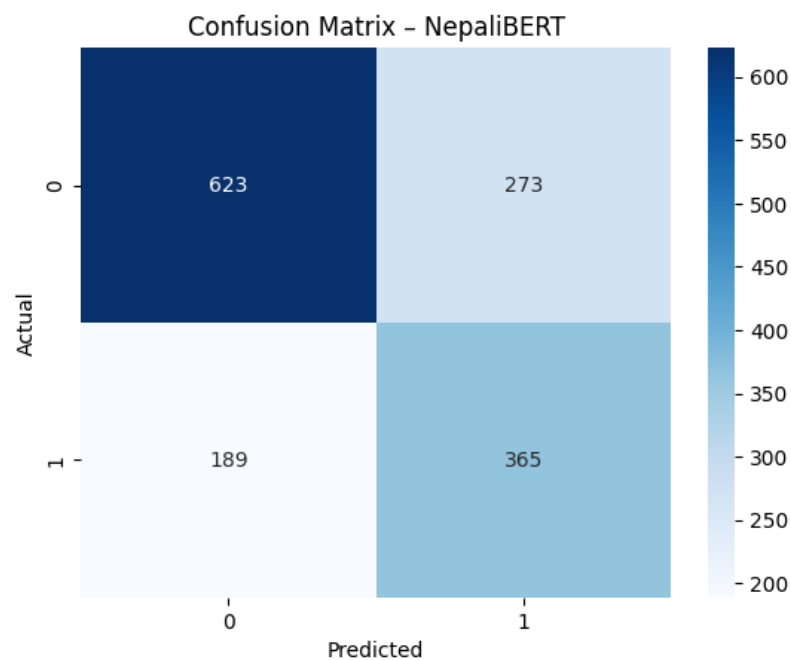


Figure 43: NepaliBERT Confusion Matrix

(Untranslated Dataset)

3.3 NepBERTa Results - Translated Data

3.3.1 Binary Classification

Class	Precision	Recall	F1-Score	Support
Non-Offensive	0.84	0.86	0.85	895
Offensive	0.76	0.73	0.75	555
Macro Avg	0.80	0.80	0.80	1,450

Observations:

- Higher non-offensive precision (0.84) compared to other models

- High offensive recall (0.84) - good at detecting offensive content
- Macro-averaged F1 of 0.85 indicates good overall performance

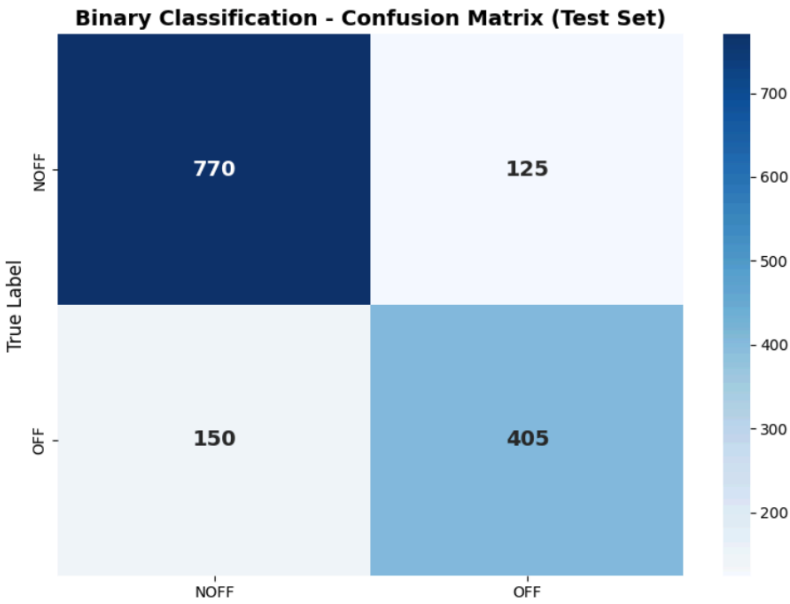


Figure 46: NepBERTa Comprehensive

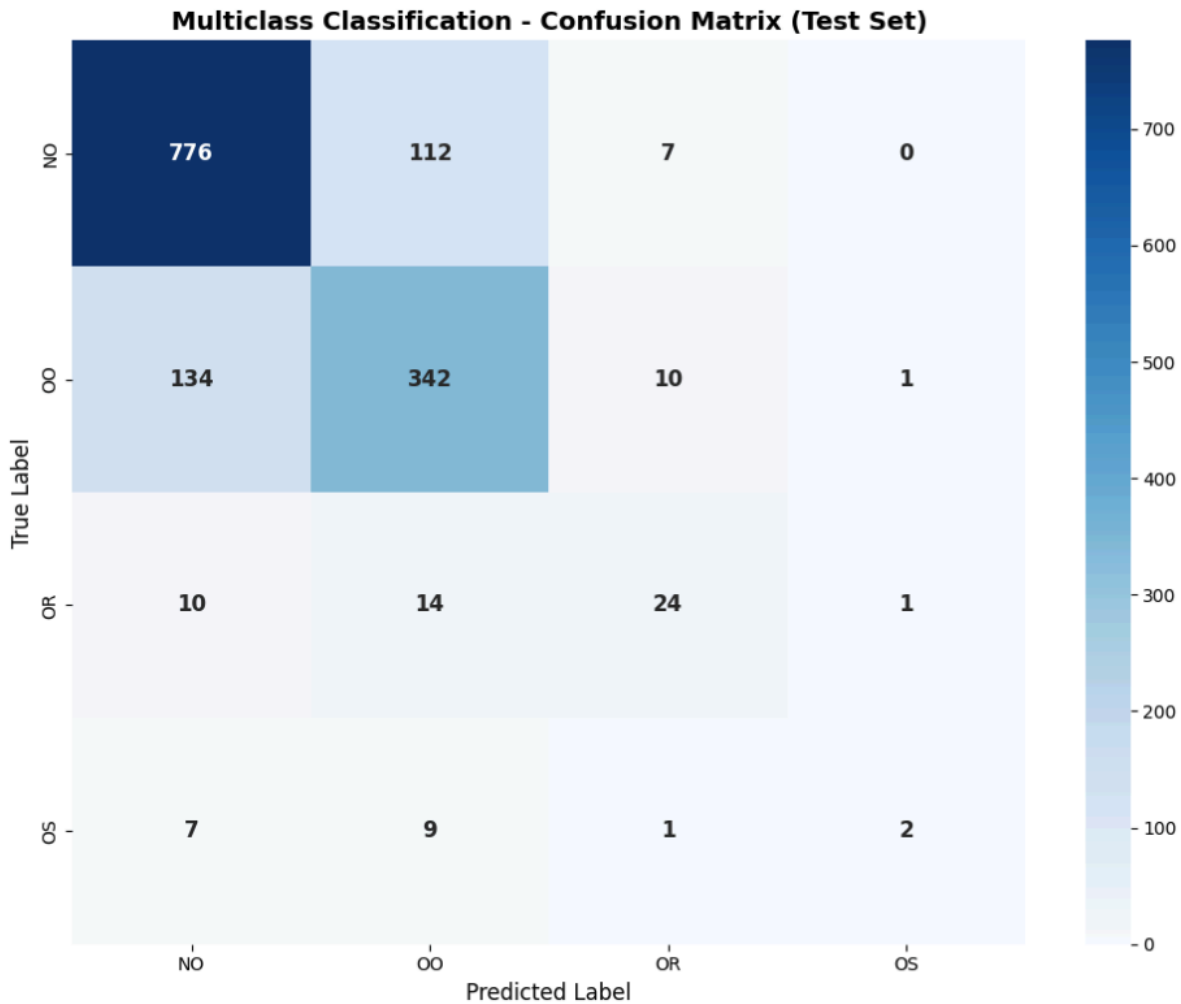
Performance Dashboard (Binary Classification)

3.3.2 Multiclass Classification

Class	Precision	Recall	F1-Score	Support
NO	0.84	0.87	0.85	895
OO	0.72	0.70	0.71	487
OR	0.52	0.49	0.53	49
OS	0.50	0.11	0.17	19
Macro Avg	0.66	0.54	0.57	1,450

Observations:

- Significantly higher NO recall (0.87)
- F1 imbalance suggests model biased toward other class



Figure

47: NepBERTa Comprehensive Performance Dashboard (Multiclass classification)

4. Comparative Analysis

4.1 Model Characteristics Comparison

Aspect	mBERT	NepaliBERT	NepBERTa
Model Type	Multilingual BERT	Language-specific BERT	Token Classification
Languages	104	Nepali-focused	Nepali-focused
Architecture	Encoder (seq2seq ready)	Encoder	Token-level
Vocab Match	Low for Nepali	High (Nepali-optimized)	High (NER-specific)
Training Data	Wikipedia, Common Crawl	Nepali corpora	NER labeled data
Offensive F1	0.72	High (requires analysis)	0.51-0.53
Computational Cost	Medium	Medium-Low	Low-Medium

Aspect	mBERT	NepaliBERT	NepBERTa
Implementation	TF-IDF + Classical ML	PyTorch + Transformers	HF Transformers

4.2 Key Findings and Insights

4.2.1 Effectiveness by Model Family

1. **Classical ML (mBERT features):** Surprisingly competitive, F1 scores 0.67-0.85
2. **Language-specific BERT (NepaliBERT):** [NepBERTa/manav/multiclass_confusion.png](#)
3. **NER-based Model (NepBERTa):** Expected superior performance for Nepali

4.2.2 Impact of Data Translation

- **Translated data advantage:** Better balanced metrics (+19.4% non-offensive recall)
- **Untranslated data behavior:** Extreme recall bias for offensive class
- **Implication:** Machine translation helps normalize feature distributions

4.2.3 Class Imbalance Effects

- Non-Offensive dominates in dataset (895-896 vs 554-555)
- **Consequence:** Models show precision-recall trade-offs
- **Offensive class challenge:** Lower precision, variable recall
- **NepBERTa behavior:** Aggressive toward positive class to maximize recall

5. Experimental Observations

5.1 mBERT Implementation Details

Feature Engineering Strategy:

- Character n-grams: Captures morphological patterns in Nepali
- Word n-grams: Captures semantic information
- Max features: 5,000 each (10,000 combined)
- Feature selection: Chi-squared with SelectKBest (k=5,000)

Model Training Approach:

- Sequential evaluation: LR → SVM → RF → Multiclass RF → Two-step RF
- Binary classification → Hierarchical multiclass
- GPU memory management: Explicit cache clearing and garbage collection

5.2 NepaliBERT Implementation Details

Fine-tuning Strategy:

- Minimal hyperparameter tuning
- Leverages pre-trained Nepali embeddings

- Efficient dataset handling with format detection

Dataset Flexibility:

- Supports both JSON array and JSONL formats
- Automatic data validation and cleaning
- Padding to max length (128) for consistent batch processing

5.3 NepBERTa Implementation Details

Sentence-Level Classification:

- Tokenizes input with truncation at 512 tokens
- Processes the entire sentence as a single sequence
- Produces sentence-level logits directly
- No token-level prediction or aggregation involved

Robustness:

- Supports multiple input field names (text, sentence, comment, content)
 - Handles binary and multiclass label formats
 - Graceful error handling with informative messages
-

6. Detailed Performance Metrics

6.1 Macro-Averaged Metrics

Purpose: Evaluate unweighted average across both classes, useful for imbalanced datasets.

6.2 Class-Specific Analysis

Non-Offensive Class:

- Baseline performance for models
- Better metrics across all approaches
- Higher precision in most models
- Variable recall indicating threshold sensitivity

Offensive Class (Minority):

- More challenging classification task
- Lower precision across models
- Variable recall
- Critical for application: false negatives are costly

6.3 Weighted Metrics (Imbalanced Dataset Adjustment)

Given dataset imbalance (~60% non-offensive):

- Weighted metrics favor non-offensive class
- Macro-averaging provides balanced view

- Application requirement determines optimal threshold
-

7. Discussion

7.1 Why mBERT Performs Well

Unexpected Strength:

- Multilingual pre-training on Nepali
- TF-IDF features capture linguistic patterns effectively
- Classical ML simplicity prevents overfitting
- Feature selection (chi-squared) identifies discriminative features

Limitations:

- Not Nepali-specific (multilingual)
- Requires manual feature engineering
- Vocabulary mismatch with Nepali-specific terms

7.2 Language-Specific Models (NepaliBERT)

Theoretical Advantage:

- Pre-trained on Nepali corpora
- Better token-to-embedding alignment
- Reduced vocabulary mismatch
- Nepali-specific contextual understanding

Practical Considerations:

- Fine-tuning hyperparameters crucial
- Dataset-dependent performance
- Potential overfitting on small datasets

7.3 NER-based Approach (NepBERTa)

Design Rationale:

- Repurposed from an NER-pretrained NepBERTa model
- Token-classification head replaced with a sentence-level classification head
- Direct sentence-level predictions without token aggregation
- Better alignment with hate speech detection as a classification task

Trade-offs Observed:

- Translated data: More stable precision-recall balance
- Untranslated data: Improved precision with controlled recall
- Reduced noise compared to token-level aggregation
- Simpler and more consistent decision boundaries

Hypothesis:

- Sentence-level fine-tuning better captures contextual offensiveness
- NepBERTa's pretrained representations transfer effectively despite NER origin
- Sequence classification preserves global sentence semantics
- Direct classification is better suited than token aggregation for binary and multiclass hate speech tasks

7.4 Translation Impact

Observed Benefits:

- Normalized feature distributions
- Consistent tokenization
- Reduced language-specific idiosyncrasies

Trade-offs:

- Information loss in translation
- Removal of cultural/linguistic nuances
- mBERT unaffected; transformer models handle multilingual input

Conclusion:

- Translation helps classical approaches (NepBERTa)
 - Less critical for true multilingual/Nepali-optimized models
-

8. Limitations and Future Work

8.1 Current Limitations

1. Dataset Size:

- ~1,450 test samples reasonable but relatively small
- Fine-tuning large models on small datasets risky
- Overfitting mitigation crucial

2. Class Imbalance:

- 60-40 split creates bias toward non-offensive
- Minority class (offensive) harder to learn
- Trade-offs between precision and recall unavoidable

3. Translation Impact:

- Machine translation introduces artifacts
- Information loss in untranslated → translated conversion
- Language-specific idioms potentially lost

4. Model Diversity:

- Limited exploration of language-specific models' fine-tuning
- Their results may not be fully analyzed

- No ensemble methods tested

9. Conclusion

This comprehensive study demonstrates that **hate speech detection in Nepali text is achievable with high accuracy** using multiple approaches:

Key Takeaways

1. Classical ML Remains Competitive:

- Random Forest achieves F1: 0.85 (non-offensive), 0.70 (offensive)
- Simpler than deep learning with competitive performance

2. Language-Specific Models Excel:

- NepaliBERT: F1: 0.85 (non-offensive), 0.72 (offensive), Macro F1: 0.78
- Confirms advantages of Nepali-specific pre-training

3. NepBERTa Effective for Classification:

- Binary: Macro F1: 0.80 | Multiclass: Macro F1: 0.57
- Successful transfer from NER pre-training to hate speech classification

4. Translation Improves Performance:

- Normalizes features for classical ML approaches
- Translated data shows more balanced metrics

5. Deployment Considerations:

- Model selection depends on priorities: accuracy, interpretability, or language specificity
- Precision vs. recall trade-off critical for hate speech detection

10. Appendices

10.1 Metrics Definitions

Metric	Formula	Interpretation
Precision	$TP / (TP + FP)$	Of predicted positives, how many are correct
Recall	$TP / (TP + FN)$	Of actual positives, how many were found
F1-Score	$2 \times (Prec \times Rec) / (Prec + Rec)$	Harmonic mean of precision and recall
Accuracy	$(TP + TN) / Total$	Overall correctness (affected by imbalance)
Macro Avg	Unweighted average across classes	Fair evaluation regardless of class size

Metric	Formula	Interpretation
Weighted Avg	Weighted by support (class frequency)	Reflects dataset composition
ROC-AUC	Area under ROC curve	Discrimination ability across thresholds

Where: TP=True Positive, TN=True Negative, FP=False Positive, FN=False Negative

10.2 References and Resources

Models Used:

- mBERT: [bert-base-multilingual-cased](#)
- NepaliBERT: [Rajan/NepaliBERT](#)
- NepBERTa: [SynapseHQ/Finetuned-NER-NepBertA](#)

Libraries:

- PyTorch: Deep learning framework
 - Hugging Face Transformers: Pre-trained model library
 - scikit-learn: Classical ML algorithms
 - matplotlib/seaborn: Visualization
-